Лабораторная работа №3: Функции, модули и работа с файлами

Часть 1: Функции – основа повторного использования кода

1.1 Определение и вызов функций

Простейшая функция без параметров

```
def greet():
    """Документационная строка (docstring) - описание функции"""
    print("Привет! Добро пожаловать в мир функций .")
```

Вызов функции

```
greet()
```

Функция с параметрами

```
def greet_person(name):
    print(f"Привет, {name}!")

greet_person("Анна")
```

Функция, возвращающая значение

```
def add(a, b):
    """Возвращает сумму двух чисел"""
    return a + b

result = add(5, 3)
print(f"5 + 3 = {result}")
```

Функция с несколькими возвращаемыми значениями

```
def get_min_max(numbers):
    """Возвращает минимальное и максимальное значение списка"""
    return min(numbers), max(numbers)
```

```
numbers = [4, 2, 9, 1, 5, 6]
min_val, max_val = get_min_max(numbers)
print(f"Минимум: {min_val}, Максимум: {max_val}")
```

1.2 Параметры функций: обязательные, по умолчанию, именованные

Параметры со значениями по умолчанию

```
def power(base, exponent=2):
    """Возводит base в степень exponent (по умолчанию квадрат)"""
    return base  exponent

print(power(3))      3^2 = 9
print(power(3, 3))   3^3 = 27
```

Именованные аргументы

```
def describe_person(name, age, city):
    print(f"{name}, {age} лет, проживает в {city}")
```

Можно передавать аргументы в любом порядке, указывая имена

```
describe_person(age=25, city="Москва", name="Иван")
```

Смешанный порядок (сначала позиционные, потом именованные)

```
describe_person("Мария", city="СПб", age=30)
```

1.3 Произвольное количество аргументов (*args и kwargs)

*args - кортеж произвольного количества позиционных аргументов

```
def sum_all(*args):
    """Суммирует все переданные аргументы"""
    total = 0
    for num in args:
        total += num
    return total
```

```
print(sum_all(1, 2, 3))         6
print(sum_all(10, 20, 30, 40))   100
```

kwargs - словарь произвольного количества именованных аргументов

```
def print_info(kwargs):
    """Выводит информацию в формате ключ: значение"""
    for key, value in kwargs.items():
        print(f"{key}: {value}")

print_info(name="Анна", age=25, city="Москва")
```

Задание 1.1: Напишите функцию analyze_text(text), которая принимает строку и возвращает словарь со статистикой:

Количество символов (включая пробелы)

Количество слов (разделитель - пробел)

Количество предложений (разделители '.', '!', '?')

Самое длинное слово

Часть 2: Области видимости переменных

Глобальная переменная

```
global_var = 100

def test_scope():
    Локальная переменная
    local_var = 50
    print("Внутри функции:")
    print(f"  local_var = {local_var}")
    print(f"  global_var = {global_var}")  глобальная доступна для чтения
```

```
test_scope()
print(local_var)  Ошибка! local_var не видна снаружи
```

Изменение глобальной переменной внутри функции

```
counter = 0

def increment():
    global counter  указываем, что хотим изменить глобальную переменную
    counter += 1
    print(f"Счетчик внутри функции: {counter}")

increment()
increment()
print(f"Счетчик после вызовов: {counter}")
```

Вложенные функции и замыкания

```
def outer_function(message):
    def inner_function():
        print(f"Сообщение: {message}")
    return inner_function  возвращаем функцию, не вызывая её

my_func = outer_function("Привет из замыкания!")
my_func()  вызов внутренней функции
```

Задание 2.1: Создайте функцию-счетчик make_counter(), которая возвращает другую функцию, при каждом вызове увеличивающую внутренний счетчик на 1 и возвращающую его текущее значение. Проверьте работу, создав два независимых счетчика.

Часть 3: Модули – расширяем возможности

3.1 Импорт встроенных модулей

```
import math
import random
from datetime import datetime, timedelta
```

```
import os
```

Модуль math

```
print("=== math ===")
print(f"Число π: {math.pi}")
print(f"Квадратный корень из 16: {math.sqrt(16)}")
print(f"Синус 30 градусов: {math.sin(math.radians(30))}")
```

Модуль random

```
print("\n=== random ===")
print(f"Случайное целое от 1 до 10: {random.randint(1, 10)}")
print(f"Случайное вещественное от 0 до 1: {random.random()}")
fruits = ['яблоко', 'банан', 'апельсин']
print(f"Случайный фрукт: {random.choice(fruits)}")
random.shuffle(fruits)
print(f"Перемешанный список: {fruits}")
```

Модуль datetime

```
print("\n=== datetime ===")
now = datetime.now()
print(f"Текущая дата и время: {now}")
print(f"Сегодня: {now.strftime('%d.%m.%Y')}")
tomorrow = now + timedelta(days=1)
print(f"Завтра: {tomorrow.strftime('%d.%m.%Y')}")
```

Модуль os (работа с операционной системой)

```
print("\n=== os ===")
print(f"Текущая рабочая директория: {os.getcwd()}")
print(f"Список файлов в текущей папке: {os.listdir('.')}")
```

3.2 Создание собственного модуля

Создайте файл my_utils.py в той же папке, что и основной скрипт:

my_utils.py

```
def is_even(n):
    """Проверяет, является ли число четным"""
    return n % 2 == 0

def factorial(n):
    """Вычисляет факториал числа n"""
    if n <= 1:
        return 1
    return n * factorial(n - 1)

def reverse_string(s):
    """Возвращает перевернутую строку"""
    return s[::-1]
```

Если мы хотим, чтобы этот код выполнялся только при прямом запуске модуля,

а не при импорте, используем конструкцию:

```
if __name__ == "__main__":
    print("Модуль my_utils запущен напрямую")
    print(f"Тест: факториал 5 = {factorial(5)}")
```

Теперь в основном скрипте:

main.py

```
import my_utils

print(my_utils.is_even(10))       True
print(my_utils.factorial(5))      120
print(my_utils.reverse_string(""))  nohtyP
```

Задание 3.1: Создайте модуль string_operations.py с функциями:

count_vowels(text) – подсчет гласных

count_consonants(text) – подсчет согласных

capitalize_words(text) – делает заглавной первую букву каждого слова

remove_punctuation(text) – удаляет знаки препинания

Импортируйте модуль в другую программу и продемонстрируйте его работу.

Часть 4: Работа с файлами

4.1 Открытие и закрытие файлов. Менеджер контекста with

Запись в файл

```
with open('example.txt', 'w', encoding='utf-8') as f:
    f.write("Привет, мир!\n")
    f.write("Это вторая строка.\n")
```

Чтение всего файла

```
with open('example.txt', 'r', encoding='utf-8') as f:
    content = f.read()
    print("Содержимое файла целиком:")
    print(content)
```

Чтение построчно

```
with open('example.txt', 'r', encoding='utf-8') as f:
    print("Чтение построчно:")
    for line in f:
        print(f"Строка: {line.rstrip()}")  rstrip() удаляет символы перевода строки
```

Добавление в конец файла

```
with open('example.txt', 'a', encoding='utf-8') as f:
    f.write("Это добавленная строка.\n")

Проверка: прочитаем ещё раз
with open('example.txt', 'r', encoding='utf-8') as f:
    print("Файл после добавления:")
    print(f.read())
```

4.2 Работа с JSON

```
import json

Данные для сохранения
data = {
    'name': 'Иван',
    'age': 30,
    'hobbies': ['чтение', 'программирование', 'путешествия'],
    'is_student': False
}

Запись в JSON-файл
with open('data.json', 'w', encoding='utf-8') as f:
    json.dump(data, f, ensure_ascii=False, indent=4)

Чтение из JSON-файла
with open('data.json', 'r', encoding='utf-8') as f:
    loaded_data = json.load(f)
    print("Данные из JSON:")
    print(loaded_data)
    print(f"Имя: {loaded_data['name']}")
    print(f"Хобби: {', '.join(loaded_data['hobbies'])}")
```

4.3 Работа с CSV

```
import csv
```

Запись в CSV

```
students = [
    ['Имя', 'Возраст', 'Город'],
    ['Анна', 20, 'Москва'],
    ['Иван', 22, 'Санкт-Петербург'],
    ['Мария', 21, 'Казань']
]

with open('students.csv', 'w', newline='', encoding='utf-8') as f:
    writer = csv.writer(f)
    writer.writerows(students)
```

Чтение из CSV

```
with open('students.csv', 'r', encoding='utf-8') as f:
    reader = csv.reader(f)
    for row in reader:
        print(row)
```

Чтение как словарей (если первая строка - заголовки)

```
with open('students.csv', 'r', encoding='utf-8') as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(f"{row['Имя']}, {row['Возраст']} лет, {row['Город']}")
```

Задание 4.1: Напишите программу, которая:

Запрашивает у пользователя данные о нескольких книгах (название, автор, год).

Сохраняет их в JSON-файл.

Затем читает этот файл и выводит список книг в виде таблицы.

Добавляет возможность поиска книги по автору и вывода результата.

Часть 5: Обработка исключений

Пример: обработка ошибки деления на ноль

```
def safe_divide(a, b):
    try:
        result = a / b
    except ZeroDivisionError:
        print("Ошибка: деление на ноль!")
        return None
    else:
        Выполняется, если исключения не было
        print("Деление выполнено успешно")
        return result
    finally:
        Выполняется всегда
        print("Блок finally отработал")

print(safe_divide(10, 2))
print(safe_divide(10, 0))
```

Обработка нескольких типов исключений

```
def read_number_from_file(filename):
    try:
        with open(filename, 'r') as f:
            number = float(f.read())
    except FileNotFoundError:
        print(f"Файл {filename} не найден")
    except ValueError:
        print(f"В файле {filename} не число")
```

```
    except Exception as e:
        print(f"Произошла непредвиденная ошибка: {e}")
    else:
        print(f"Прочитано число: {number}")
        return number
    finally:
        print("Попытка чтения завершена")

read_number_from_file("nonexistent.txt")
```

Генерация исключений

```
def withdraw_money(balance, amount):
    if amount > balance:
        raise ValueError(f"Недостаточно средств. Доступно: {balance}")
    return balance - amount

try:
    new_balance = withdraw_money(100, 150)
except ValueError as e:
    print(f"Ошибка: {e}")
```

Задание 5.1: Дополните программу из задания 4.1 (книги) обработкой исключений:

При вводе года проверяйте, что это число и оно не превышает текущий год.

При работе с файлами обрабатывайте возможные ошибки (файл не найден, поврежденный JSON и т.д.).

Если пользователь вводит пустую строку для названия книги, выбрасывайте исключение с сообщением.

Итоговый проект: "Ежедневник с сохранением в файл"

Разработайте консольное приложение "Ежедневник", которое позволяет:

Добавлять задачи с указанием даты, времени и описания.

Просматривать задачи на сегодня, на завтра, на неделю.

Отмечать задачи как выполненные.

Удалять задачи.

Сохранять задачи в JSON-файл при выходе и загружать при запуске.

Все операции должны быть реализованы в виде функций. Используйте модули datetime для работы с датами, json для хранения. Обязательно добавьте обработку исключений (например, при повреждении файла).

Примерная структура программы:

```
import json
from datetime import datetime, timedelta

DATA_FILE = "tasks.json"

def load_tasks():
    """Загружает задачи из JSON-файла. Если файла нет, возвращает пустой список."""
    try:
        with open(DATA_FILE, 'r', encoding='utf-8') as f:
            return json.load(f)
    except FileNotFoundError:
        return []
    except json.JSONDecodeError:
        print("Файл с задачами поврежден. Начинаем с пустого списка.")
        return []
```

```
def save_tasks(tasks):
    """Сохраняет задачи в JSON-файл."""
    with open(DATA_FILE, 'w', encoding='utf-8') as f:
        json.dump(tasks, f, ensure_ascii=False, indent=4)

def add_task(tasks):
    """Добавляет новую задачу."""
    print("\n--- Добавление задачи ---")
    description = input("Описание задачи: ")
    if not description:
        print("Описание не может быть пустым!")
        return

    date_str = input("Дата (ДД.ММ.ГГГГ) или оставьте пустым для сегодня: ").strip()
    if not date_str:
        date = datetime.now().date()
    else:
        try:
            date = datetime.strptime(date_str, "%d.%m.%Y").date()
        except ValueError:
            print("Неверный формат даты! Используется сегодняшняя.")
            date = datetime.now().date()

    time_str = input("Время (ЧЧ:ММ) или оставьте пустым: ").strip()
    time = time_str if time_str else None

    task = {
        'id': len(tasks) + 1,
        'description': description,
        'date': date.strftime("%d.%m.%Y"),
        'time': time,
        'completed': False
```

```
    }
    tasks.append(task)
    print("Задача добавлена!")

def show_tasks(tasks, filter_date=None):
    """Показывает задачи. Если filter_date указана, только за эту дату."""
    if not tasks:
        print("Нет задач.")
        return

    Сортировка по дате и времени
    sorted_tasks = sorted(tasks, key=lambda t: (datetime.strptime(t['date'], "%d.%m.%Y"), t['time'] or "00:00"))

    for task in sorted_tasks:
        task_date = datetime.strptime(task['date'], "%d.%m.%Y").date()
        if filter_date and task_date != filter_date:
            continue

        status = "✓" if task['completed'] else "✗"
        time_str = f" {task['time']}" if task['time'] else ""
        print(f"[{status}] {task['id']}. {task['date']}{time_str} - {task['description']}")

def mark_completed(tasks):
    """Отмечает задачу выполненной."""
    task_id = int(input("Введите ID задачи для отметки: "))
    for task in tasks:
        if task['id'] == task_id:
            task['completed'] = True
            print("Задача отмечена выполненной.")
            return
    print("Задача с таким ID не найдена.")
```

```
def delete_task(tasks):
    """Удаляет задачу."""
    task_id = int(input("Введите ID задачи для удаления: "))
    for i, task in enumerate(tasks):
        if task['id'] == task_id:
            del tasks[i]
            print("Задача удалена.")
            return
    print("Задача с таким ID не найдена.")

def main():
    tasks = load_tasks()

    while True:
        print("\n=== Ежедневник ===")
        print("1. Показать все задачи")
        print("2. Показать задачи на сегодня")
        print("3. Показать задачи на завтра")
        print("4. Добавить задачу")
        print("5. Отметить выполненной")
        print("6. Удалить задачу")
        print("7. Выход")

        choice = input("Выберите действие: ")

        if choice == '1':
            show_tasks(tasks)
        elif choice == '2':
            show_tasks(tasks, datetime.now().date())
        elif choice == '3':
            tomorrow = datetime.now().date() + timedelta(days=1)
            show_tasks(tasks, tomorrow)
```

```
        elif choice == '4':
            add_task(tasks)
        elif choice == '5':
            mark_completed(tasks)
        elif choice == '6':
            delete_task(tasks)
        elif choice == '7':
            save_tasks(tasks)
            print("До свидания!")
            break
        else:
            print("Неверный ввод, попробуйте снова.")

if __name__ == "__main__":
    main()
```

Задание для самостоятельной доработки:

Добавьте возможность редактирования задачи.

Реализуйте поиск по описанию.

Добавьте категории задач и фильтрацию по ним.

Сделайте экспорт задач в текстовый файл.